



20.10.21 13:06

Tutorial

Language Sequence

Modelling



Jetic Gū
CMPT413/713

Overview

- Focus: Language Modelling
- Architecture: Any Neural Sequence Model
- Core Ideas:
 1. Review: Neural LM
 2. Theory: Neural LM encoder
 3. Theory: Neural LM decoder
 4. Sequence to Sequence

What you have seen so far

- Neural Language Models
 - With Generative Objectives
 - With Discriminative Objectives

RNN Encodes a Sentence

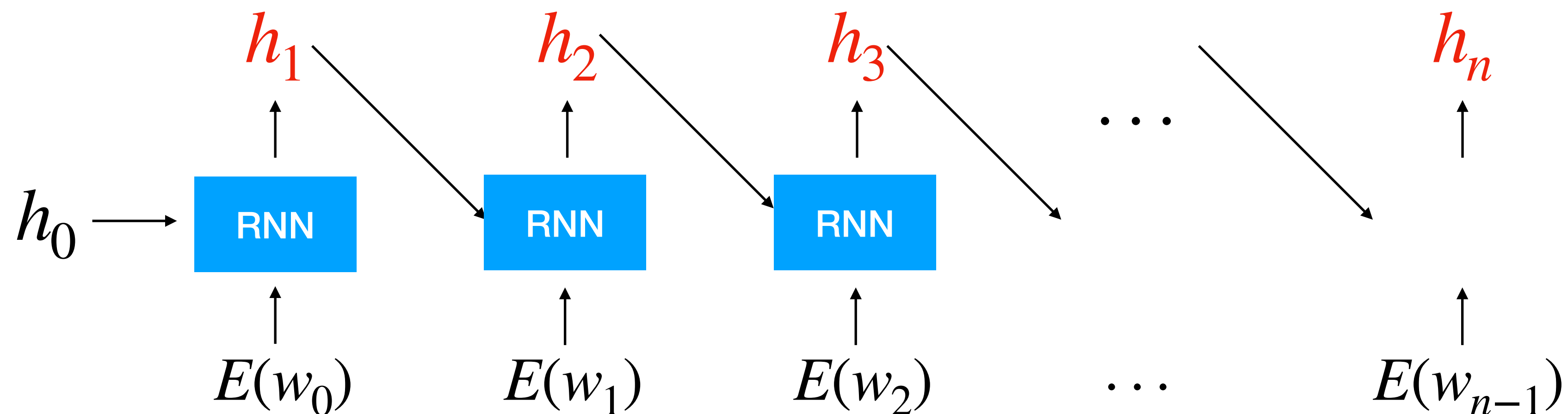
- Any RNN encodes a sentence $S = w_0, w_1, w_2, \dots, w_{n-1}$

 h_0

RNN

RNN Encodes a Sentence

- Any RNN encodes a sentence $S = w_0, w_1, w_2, \dots, w_{n-1}$



Recurrent Neural Network

- Any RNN
 - For input sequence $X = x_0, x_1, \dots, x_{n-1}$
 - Vector h_{i-1} holds information about $x_0, \dots, x_{i-1}$
 - $h_i = RNN(h_{i-1}, x_i) = MLP([h_{i-1}; x_i])$
where $[a; b]$ is the concatenation of vectors a and b
 - h_0 is part of your parameter that gets trained with the rest of your model

NLM Usage

- Neural Networks are feature mapping functions



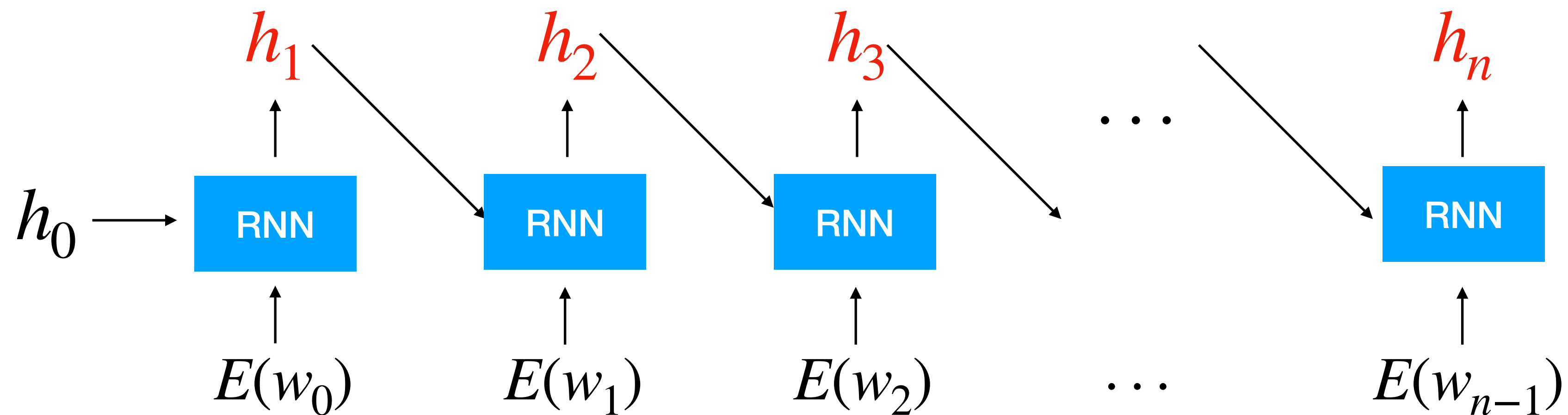
NLM Usage

- Neural Networks are feature mapping functions



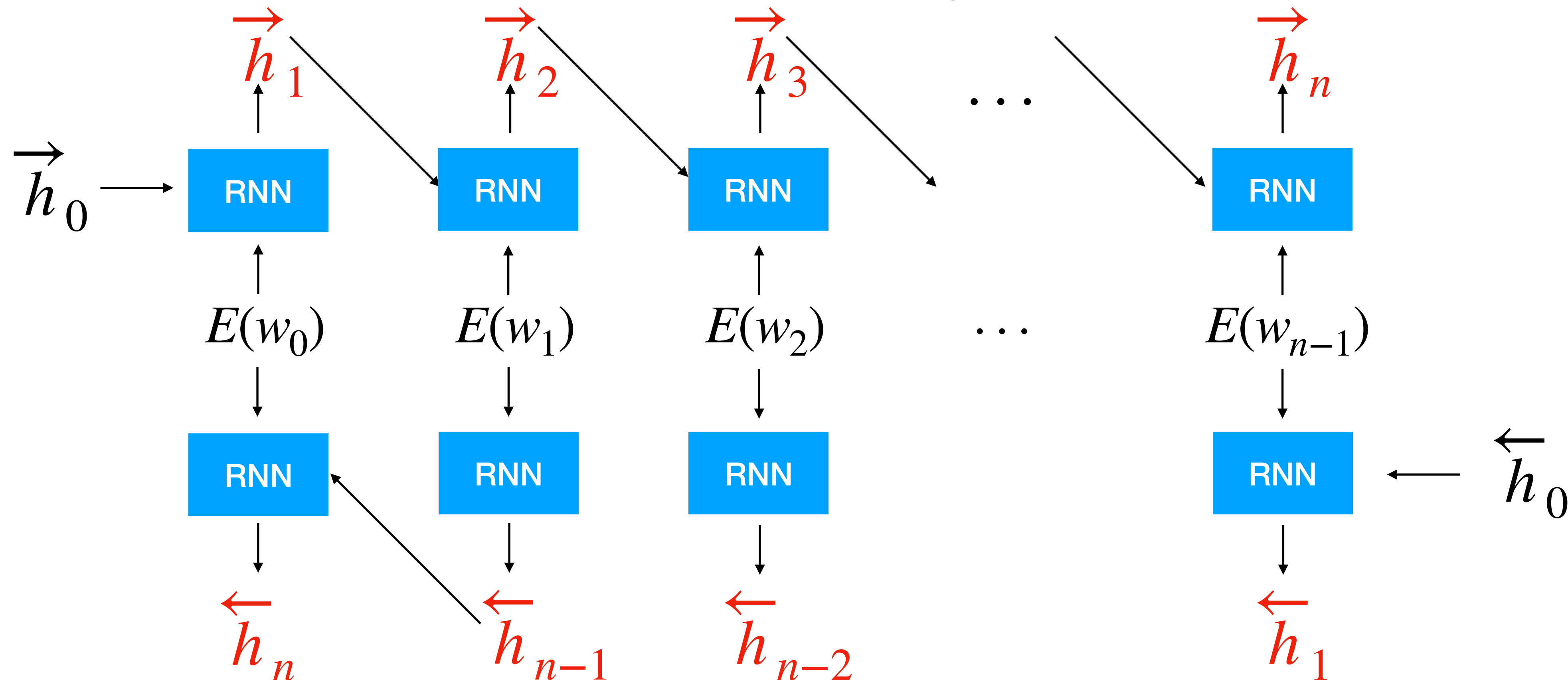
Discriminative Training Sentence Classification

- Any RNN encodes a sentence $S = w_0, w_1, w_2, \dots, w_{n-1}$



Discriminative Training Sentence Classification

- Bidirectional RNN encodes a sentence $S = w_0, w_1, w_2, \dots, w_{n-1}$



Discriminative Training

Sentence Classification

- Bidirectional RNN encodes a sentence $S = w_0, w_1, w_2, \dots, w_{n-1}$
- Forward RNN: encodes sentence from left-to-right: $\vec{h}_{1:n}$
- Backward RNN: encodes sentence from right-to-left: $\overleftarrow{h}_{1:n}$
- Representation for each position $1 < i < n$: $[\vec{h}_i; \overleftarrow{h}_{n-i+1}]$
- Representation for entire sentence: $[\vec{h}_n; \overleftarrow{h}_n]$
- RNNs are found to be biased towards the last input it receives. By using BiRNN, you get more balanced representation

Discriminative Training

Sentence Classification

- Sentence Classification projection, say into 3 classes [pos, neutral, neg]
- $y = \text{softmax}(\sigma(W[\vec{h}_n; \overleftarrow{h}_n] + b))$
where W and b projects the sentence representation into a 3 dimensional vector
- $Loss = -\log(y_0)$ negative log loss for positive sample
 $Loss = -\log(y_1)$ negative log loss for neutral sample
 $Loss = -\log(y_2)$ negative log loss for negative sample

RNN/Transformer

- RNN and Transformers can transform a natural language sentence into feature representations
- This process is called **Encoding**
- The process of generating predictions from encoded representations using whatever method, is called **Interpreting**, or more commonly, **Decoding**
- Encoder representations are usually very task specific

NLM Usage

- Neural Networks are feature mapping functions



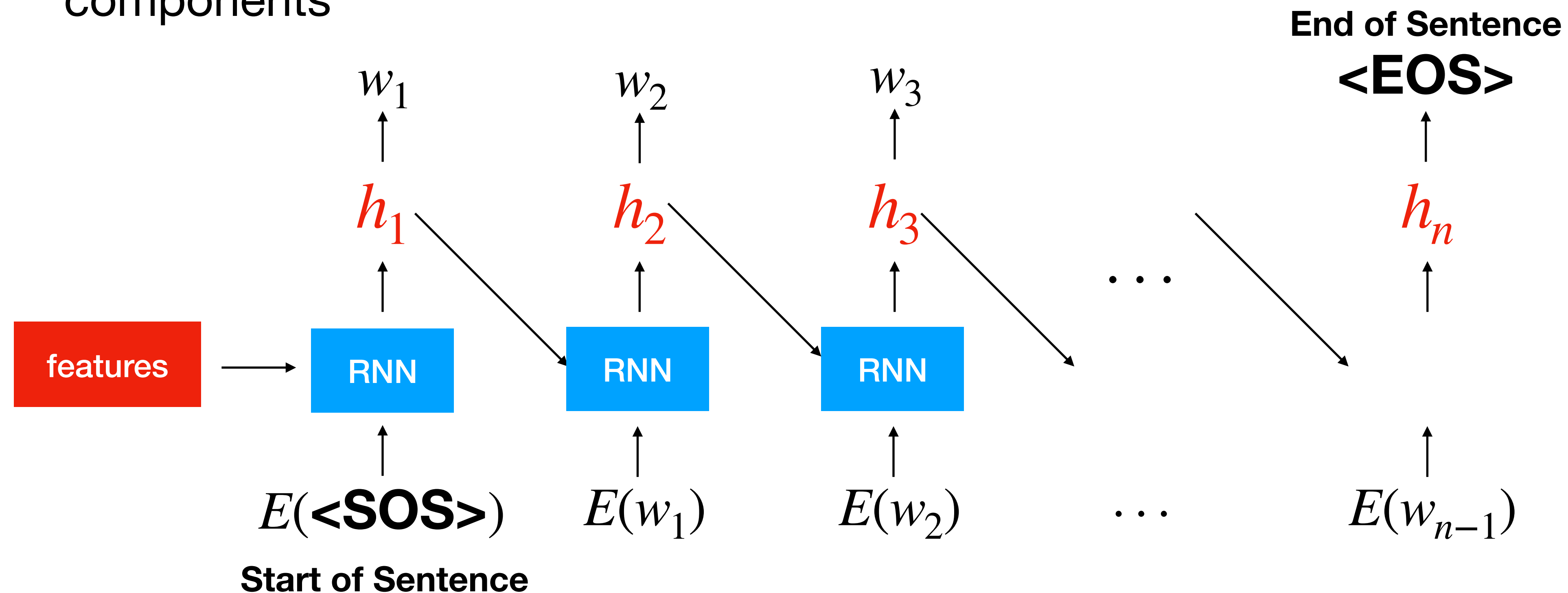
NLM Usage

- Neural Networks are feature mapping functions



Vanilla Generative RNN

- Generative RNNs can be guided by features provided by other neural components



Vanilla Generative RNN

- Initialise your Generative RNN with **input feature vector**
- For the first step, **feed in a special token <SOS>**, to start predicting the first word
- For the i -th step, **predict w_i from h_i** by mapping function (like MLP)
transform h_i to dictionary-sized vector, and choose the word with the highest score
- For the $i + 1$ -th step, **feed $E(w_i)$ back into the RNN for h_{i+1}**
- **Stop generation** when a special token **<EOS>** is spit out
- Train with negative log loss

SoTA Generative RNN

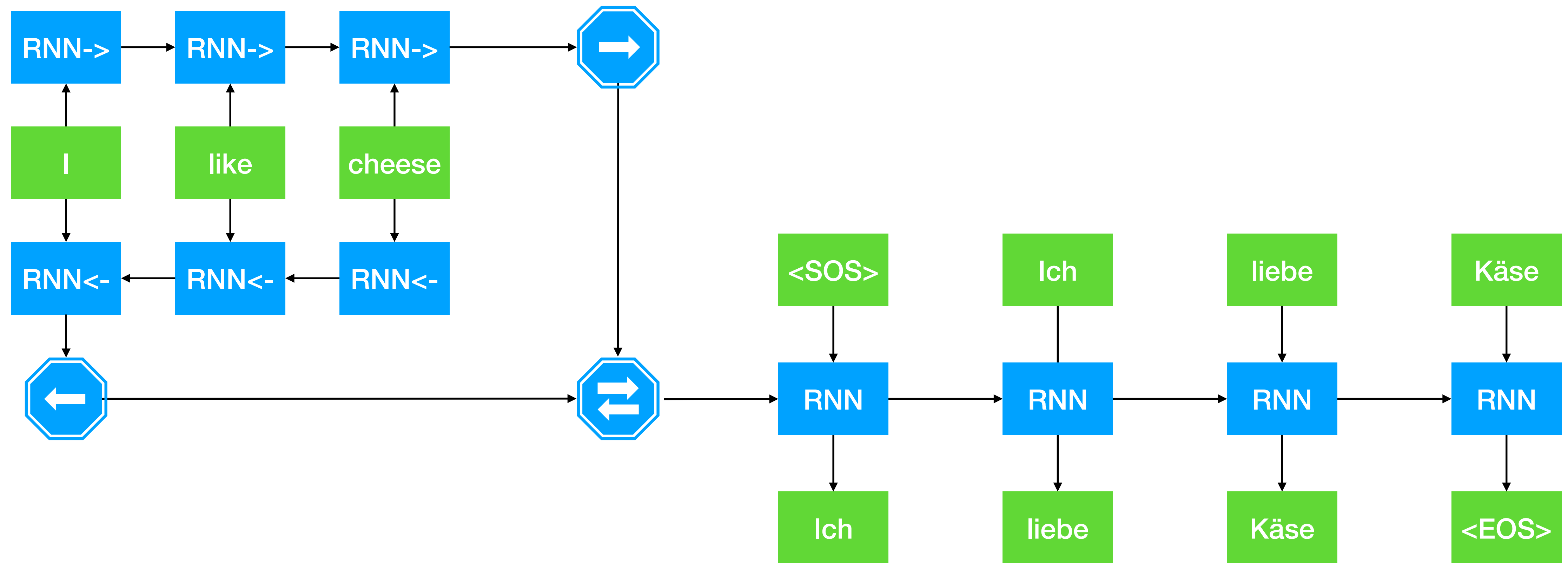
- What has changed?
 - Use transformers instead of LSTM (not necessarily better)
 - Use attention mechanism to guide RNN instead of initial feature vector
 - Use hybrid mechanism such as copy and pointer-generator*
- The generative mechanism has not changed a lot in the past 6 years

Vanilla Seq2Seq

- Encoder-Decoder Architecture
 - Encoder transforms source input into feature representation
 - Decoder transforms feature representation into target output



Vanilla Seq2Seq Translation



Technical

Seq2Seq Application

- Text-to-Text
 - Machine Translation: src in French, tgt in English
 - Summarisation: src in paragraph, tgt in sentence
 - Question Answering: src is question, tgt is answer (not very good)
 - Anything where the source is a sequence and target also sequence

Seq2Seq Improvements

- Encoder and Decoder: Use transformers instead of RNN
- Between Encoder and Decoder: use attention
- Other stuff:
 - Multiple encoder, one decoder
 - One encoder, multiple decoder
 - Multiple encoder, multiple decoder, etc.